National University of Computer & Emerging Sciences, Karachi

School of Computing (BSCS, BSSE, BSCY, BSAI)

PF-Assignment # 2, Fall2026

Course Code: CS-1002	Course Name: Programming Fundamentals
Open Date: 13/October/2026	Due Date: 22/October/2026

Instructions:

A. You must never share your assignment with anyone. If any part of your assignment is found to be copied, then you will be awarded 0 marks as per the university policy.

B. The assignment must be submitted before the deadline and only on GCR.

C. All Programs should be submitted in softcopy and should have .c file

Question # 1:

Cinema Hall Seat Booking Management System

A cinema hall wants to develop a simple seat booking management system to manage seat availability and allocate seats to groups of customers. The seating arrangement is stored in a two-dimensional array, where **0 represents an available seat and 1 represents a booked seat**.

The system should help cinema staff view the seating arrangement, check seat availability, and allocate consecutive seats to groups of customers. To provide customers with a better viewing experience, the system prioritizes rows closest to the middle of the hall.

Write a **C program using a two-dimensional array and the main() function only** to perform the following tasks:

1. **Input Seating Plan:** Input the number of rows and the number of seats per row. Input the current seating status of every seat, where 0 indicates an available seat and 1 indicates a booked seat.
2. **Display Seating Arrangement:** Display the complete seating plan, showing _ for available seats and X for booked seats. Display row numbers and seat numbers starting from 0.
3. **Seat Availability Report:** Display the number of available seats in each row and calculate the total number of available seats in the entire hall.
4. **Group Booking:** Input the number of customers, k, who want to sit together. Find k consecutive available seats in the same row.
5. **Seat Allocation Rules:** Follow these rules when searching for seats:
    - Start searching from the middle row, calculated as rows / 2 using integer division.
    - Search the remaining rows in increasing order of distance from the middle row.
    - If two rows are equally distant from the middle row, search the row with the smaller row number first.
    - Within each row, select the first available block of k consecutive seats, starting from the lowest seat number.
    - Stop searching as soon as the first suitable block is found.
6. **Confirm Booking:** If a suitable block is found, mark the allocated seats as booked by changing their values to 1. Display the allocated row number and seat numbers,

then display the updated seating arrangement and the remaining number of available seats.

7. **Handle Unsuccessful Bookings:** If no row contains k consecutive available seats, display an appropriate message and leave the seating plan unchanged.

Requirements:

- Implement the complete program inside main() without defining or calling any user-defined functions.
- Validate the group size to ensure that it is greater than zero and does not exceed the number of seats in a row.
- Update the seating plan only when a suitable block of consecutive available seats has been found.
- Display all row and seat numbers starting from 0.

Sample input (5 rows, 8 seats, k = 3):

```
0 0 1 0 0 0 0 1
0 1 1 0 0 0 1 0
1 0 0 0 1 0 0 0
0 0 0 0 0 1 1 0
1 1 0 0 0 0 0 0
```

Sample output:

```
Free seats per row: 6 5 6 6 6   Total free: 29

Group of 3 seated in Row 2, Seats 1 to 3

     0 1 2 3 4 5 6 7
R0   _ _ X _ _ _ _ X
R1   _ X X _ _ _ X _
R2   X X X X X _ _ _
R3   _ _ _ _ _ X X _
R4   X X _ _ _ _ _ _
```

Hint: For each row, use a loop that counts how many free seats in a row you have seen without a break. When the count reaches k, you have found a block, and its starting seat is (current seat − k + 1). Reset the count whenever you meet a booked seat.

Question # 2:

Two different positive numbers are called an amicable pair if the sum of the proper divisors of the first number equals the second number, and the sum of the proper divisors of the second number equals the first number. Proper divisors of a number are all of its divisors except the number itself.

For example, the proper divisors of 220 are 1, 2, 4, 5, 10, 11, 20, 22, 44, 55 and 110. Their sum is 284. The proper divisors of 284 are 1, 2, 4, 71 and 142, and their sum is 220. So (220, 284) is an amicable pair.

Part A.

Write a C program that inputs a number n and displays all amicable pairs in which both numbers are less than or equal to n. Follow these rules:

1. Display each pair only once, with the smaller number first.
2. Do not display a number that is equal to the sum of its own proper divisors (for example 6 or 28). The two numbers of a pair must be different.
3. Write a function that returns the sum of the proper divisors of a number, and use it in your solution.

4. At the end, display how many pairs were found. If there are none, display a suitable message.

Sample input: n = 3000

Sample output:

```
Amicable pairs up to 3000:
(220, 284)
(1184, 1210)
(2620, 2924)
Total pairs: 3
```

Part B.

Draw a trace table on paper for the input 220. The table must show, for each value of the loop variable i that you test, whether i divides 220 and the running sum of divisors. Then show how the program checks the second number, 284, and decides that (220, 284) is a pair. Include a photo or scan of the trace table in your document.

Hint: For a number x you only need to test divisors up to half of x, because no proper divisor of x is larger than x / 2. For a faster version, test up to the square root of x and add both divisors of each pair found (take care not to add a square-root divisor twice and not to include x itself).

Question # 3:

Greenhouse Temperature Monitoring System

A greenhouse manager uses multiple temperature sensors to monitor the temperature inside a greenhouse. Each sensor records the temperature in degrees Celsius at the end of every hour. The readings are stored in a two-dimensional array, where each row represents a sensor and each column represents an hourly reading.

The manager wants to analyze the recorded temperatures to identify potentially dangerous conditions for the plants.

Write a **C program using functions and a two-dimensional array** to perform the following tasks:

1. **Input Data:** Input the number of sensors and the number of hourly readings from the user. Store the temperature readings for every sensor and every hour in a two-dimensional array.
2. **Input Function:** Write a function to input all temperature readings into the two-dimensional array.
3. **Sensor Analysis Function:** Write a function that receives the readings of a sensor and calculates the following:
    - Total of all temperature readings.
    - Average temperature, displayed up to two decimal places.
    - Highest and lowest temperature readings.
    - Number of critical readings (temperature of 40°C or above).
    - Number of warning readings (temperature between 35°C and 39°C, inclusive).
    - Longest streak of consecutive hours during which the temperature remained at 35°C or above.
4. **Individual Reports:** Display a separate report for each sensor containing all the calculated statistics.
5. **Highest Average Function:** Write a separate function to determine and return the sensor number with the highest average temperature.

6. **Final Report:** Display the sensor number with the highest average temperature along with its average reading.

Requirements:

- Use a two-dimensional array to store the temperature readings.
- Use separate user-defined functions for inputting readings, analyzing a sensor's readings, and identifying the sensor with the highest average temperature.
- Pass the required array elements and values as function arguments.
- Use appropriate return values wherever necessary.
- Display all average temperatures with exactly two decimal places.
- Assume that at least one sensor and one hourly reading will be entered by the user.

Sample input (3 sensors, 6 readings each):

```
Sensor 1: 28 31 36 41 43 38
Sensor 2: 25 27 29 30 32 34
Sensor 3: 37 39 40 42 36 35
```

Sample output:

```
Sensor 1: Total=217  Avg=36.17  Max=43  Min=28
          Critical=2  Warning=2  Longest hot streak=4
Sensor 2: Total=177  Avg=29.50  Max=34  Min=25
          Critical=0  Warning=0  Longest hot streak=0
Sensor 3: Total=229  Avg=38.17  Max=42  Min=35
          Critical=2  Warning=4  Longest hot streak=6

Highest average: Sensor 3 (38.17)
```

Note: A streak is a run of readings that are next to each other in the same row. A reading below 35 ends the streak and the count starts again.

Question # 4:

Bakery Festival Sales Management System

A bakery participates in a food festival that runs for several days. Each day, the bakery receives a different number of customer orders, and each order has a different sale amount. The owner wants to generate a sales summary to evaluate the bakery's performance throughout the festival.

Write a **C program using loops, arrays, and user-defined functions** to perform the following tasks:

1. Input Festival Duration

Ask the user to enter the number of festival days. The value must be between 1 and 7. If an invalid value is entered, keep asking until a valid value is provided.

2. Daily Order Processing

For each festival day:

- Input the number of customer orders received.
- Input the amount in rupees for each order. Each amount must be greater than zero. If an invalid amount is entered, ask again for the same order.
- Calculate and display the total sales, average order amount (two decimal places), and highest order amount for that day.

3. Festival Sales Summary

After processing all days, display the total sales for the entire festival and the day number with the highest total sales. If multiple days have the same highest sales, display the earliest such day.

4. Daily Sales Bar Chart

Display a bar chart of daily sales using asterisks (*). Print one asterisk for every Rs. 100 of daily sales, ignoring any remaining amount below Rs. 100.

Programming Requirements

Use a one-dimensional array to store the total sales for each festival day. Implement the program using the following user-defined functions with the specified parameters and return types:

1. **int getValidDays()**
   - Takes no arguments.
   - Inputs and validates the number of festival days (1–7).
   - Returns the valid number of days.
2. **float calculateDailySales(int dayNumber)**
   - Receives the current festival day number as an integer argument.
   - Inputs the number of orders and the amount of each order, validating that every amount is greater than zero.
   - Calculates and displays the day's total sales, average order amount, and highest order amount.
   - Returns the total sales for that day as a float.
3. **int findHighestSalesDay(float dailySales[], int numberOfDays)**
   - Receives the one-dimensional array dailySales[], containing the total sales for each day, and the integer numberOfDays, representing the number of festival days.
   - Finds the day with the highest total sales.
   - Returns the **day number starting from 1**. If multiple days have the same highest sales, returns the earliest day.
4. **void displayBarChart(float dailySales[], int numberOfDays)**
   - Receives the dailySales[] array and the integer numberOfDays.
   - Displays the sales bar chart for every day, printing one asterisk for each complete Rs. 100 in that day's total sales.

Additional requirements:

- In main(), call the functions in an appropriate sequence and store each day's returned sales total in dailySales[].
- Display all average order amounts with exactly two decimal places.
- Assume that the number of orders entered for each day is at least one.

Sample Input:

```
Number of days: 3
Day 1 - Number of orders: 4
Order amounts: 250 400 150 300
Day 2 - Number of orders: 3
Order amounts: 500 700 200
Day 3 - Number of orders: 5
Order amounts: 100 200 150 350 250
```

Sample Output:

```
Total sales: Rs. 1050.00
Average order: Rs. 210.00
Highest order: Rs. 350.00

===== FESTIVAL SUMMARY =====
Total festival sales: Rs. 3550.00
Day with highest sales: Day 2
Highest daily sales: Rs. 1400.00

===== DAILY SALES BAR CHART =====
Day 1: ********** (Rs. 1100.00)
Day 2: ************** (Rs. 1400.00)
Day 3: ********** (Rs. 1050.00)
```

Question # 5:

University Library Management System

A university library wants to develop a menu-driven application to manage its books, registered members, borrowing records, and late return fines. The system should allow members to borrow and return books while maintaining updated stock and fine records.

Write a **C program using arrays, loops, conditional statements, and user-defined functions** to perform the following tasks.

1. Member Registration

Allow a member to enter their name, student ID, and membership type (Student or Faculty).

- Store the member's details for subsequent operations.
- Student members are allowed to keep books for 14 days, while Faculty members are allowed to keep books for 30 days.
- Maintain the member's borrowing record and fine details.

2. Display Available Books

Display all books in a tabular format containing the book ID, title, and number of available copies.

ID	Title	Copies
101	Programming in C	5
102	Data Structures	3
103	Database Systems	0
104	Computer Networks	4

3. Borrow a Book

Ask the member to enter a book ID and the number of days they intend to keep the book.

- If the book ID is invalid or no copies are available, display an appropriate message.
- A member can borrow a maximum of three books at a time.
- If the request is valid, record the borrowed book and decrease its available copies by one.

4. Return a Book

Ask the member to enter the book ID and the actual number of days the book was kept.

- Verify that the member has borrowed the specified book.

- Increase the available copies by one after a valid return.
- If the book is returned after the permitted borrowing period, calculate the late fine:
  - Student: Rs. 20 per extra day.
  - Faculty: Rs. 10 per extra day.
- Ask the member to enter the membership code LIB2026. If the code is correct, apply a 50% waiver to the calculated fine.
- Update the member's fine record and remove the returned book from the current borrowing list.

5. Display Member Record

Display the member's name, student ID, membership type, list of currently borrowed books, total fine charged before the waiver, and final fine payable after the waiver.

6. Exit

Terminate the program when the user selects Exit.

Programming Requirements

User-Defined Functions

Implement the following functions with the specified arguments and responsibilities:

1. **void displayMenu()**
   - Takes no arguments.
   - Displays all menu options.
2. **void registerMember(char name[], int *studentID, int *memberType)**
   - Receives the member's name array and pointers to the student ID and membership type.
   - Inputs and stores the member's registration details.
3. **void displayBooks(int bookIDs[], char bookTitles[][50], int availableCopies[], int totalBooks)**
   - Receives the book ID array, title array, available-copy array, and total number of books.
   - Displays all books and their available copies in a table.
4. **void borrowBook(int bookIDs[], char bookTitles[][50], int availableCopies[], int totalBooks, int borrowedBookIDs[], int *borrowedCount, int memberType)**
   - Receives the book inventory arrays, total number of books, borrowed-book array, a pointer to the current borrowing count, and the member type.
   - Validates the book ID, availability, and borrowing limit.
   - Records a successful borrowing and updates the available copies.
5. **float returnBook(int bookIDs[], int availableCopies[], int totalBooks, int borrowedBookIDs[], int *borrowedCount, int memberType, float *totalFine)**
   - Receives the book inventory arrays, total number of books, borrowed-book array, a pointer to the borrowing count, member type, and a pointer to the cumulative fine.
   - Validates the return, updates the available copies, calculates the late fine, checks the membership code, and applies the waiver when valid.
   - Updates the borrowing record and returns the fine payable for the current return.
6. **void displayMemberRecord(char name[], int studentID, int memberType, int borrowedBookIDs[], int borrowedCount, float totalFine, float fineAfterWaiver)**
   - Receives the member's details, borrowed-book IDs, current borrowing count, cumulative fine before waivers, and final fine payable after waivers.
   - Displays the member's information and current borrowing record.
7. **int main()**

- Initializes the book inventory and member-record variables.
- Displays the menu repeatedly using a do-while loop.
- Uses a switch statement to call the appropriate function for each menu option.
- Continues until the user chooses Exit.

C. Additional Requirements

- Handle invalid menu choices and membership types.
- Handle invalid book IDs and unavailable books.
- Enforce the maximum limit of three borrowed books.
- Prevent returning a book that the member has not borrowed.
- Ensure the number of days entered is positive.
- Apply the fine waiver only when the correct membership code is entered.
- Maintain accurate borrowing records, available copies, and fine amounts throughout the program.